

BoiteAVoisins

Projet DevSecOps / AWS

Conception et deployment d'une infrastructure cloud

Rapport technique - Partie AWS

Region us-east-1 - Architecture ECS Fargate + ALB + RDS

Clara SION SAUNIER, Eva DOUMBE NGANGUE, Onur GENC - APPING2

Juin 2026

1. Introduction et perimetre

Ce rapport documente la partie AWS du projet DevSecOps. L'objectif n'est pas la qualite du code applicatif, mais l'ensemble de l'infrastructure mise en place autour de l'application : reseau, base de donnees, compute, securite, secrets, observabilite, et leur automatisation via Terraform.

L'application support, BoiteAVoisins, est une plateforme de prets d'objets entre voisins. Elle se compose d'une API REST Node.js / Express (port 4000, base PostgreSQL) et d'un frontend React (Vite). Le choix de l'application est secondaire : elle sert de charge utile a deployer.

La repartition initiale du projet prevoyait trois roles : Eva (reseau, base de donnees, secrets), Clara (compute, frontend) et Onur (CI/CD, conteneurisation, scans). Chaque membre disposant de son propre compte AWS Academy Learner Lab (comptes isolees), l'ensemble de l'infrastructure AWS a finalement ete deploye dans un compte unique afin d'obtenir une stack coherente et fonctionnelle de bout en bout. Le code Terraform etant entierement parametre (aucun identifiant de ressource en dur), il est portable d'un compte a l'autre.

1.1 Technologies utilisees

Domaine	Technologie
Infrastructure as Code	Terraform (~> 5.0), backend S3 + verrou DynamoDB
Reseau	VPC, subnets multi-AZ, IGW, NAT Gateway
Compute	ECS Fargate (serverless containers) + Auto Scaling
Repartition de charge	Application Load Balancer (ALB)
Base de donnees	RDS PostgreSQL 16.9 (chiffree, multi-AZ desactive en staging)
Conteneurs	Docker, registre ECR (scan a la volee)
Secrets	AWS Secrets Manager (mot de passe RDS + secret JWT)
Frontend	S3 (website hosting statique)
Observabilite	CloudWatch Logs, Container Insights, alarme + SNS

2. Architecture globale

L'architecture repose sur une separation reseau stricte en trois niveaux (public, applicatif prive, donnees prive) repartis sur deux zones de disponibilite (us-east-1a et us-east-1b), conformement a l'exigence d'un reseau multi-subnet multi-AZ.

2.1 Schema logique des flux

Le flux nominal, du client jusqu'a la base de donnees :

```

Internet
| (HTTP 80)
v
[ ALB ] -- subnets PUBLICS (AZ a + b)
| (HTTP 4000, autorise uniquement depuis le SG de l'ALB)
v
[ ECS Fargate x2 ] -- subnets PRIVES APP (AZ a + b)
| (PostgreSQL 5432, autorise uniquement depuis le SG ECS)
v
[ RDS PostgreSQL ] -- subnets PRIVES DATA (AZ a + b)

[ S3 website ] --> Frontend React (statique)
Secrets Manager --> mot de passe RDS + secret JWT (injectes dans ECS)
  
```

2.2 Principe de moindre privilege (least privilege)

Les flux reseau sont volontairement restreints au strict necessaire. Chaque Security Group n'autorise en entree que la source legitime :

Security Group	Entree autorisee	Source
SG ALB	HTTP 80	0.0.0.0/0 (Internet)
SG ECS	TCP 4000	Uniquement le SG de l'ALB
SG RDS	TCP 5432	Uniquement le SG ECS

Point notable : le Security Group ECS a ete resserre par rapport a la version initiale (qui autorisait tout le VPC 10.0.0.0/16 sur le port 4000). Desormais, seul le trafic provenant du Security Group de l'ALB est accepte. La base de donnees, elle, n'est joignable que depuis les taches ECS. Aucune ressource applicative ou de donnees n'est exposee directement a Internet.

3. Detail des composants AWS

3.1 Reseau (module vpc)

Un VPC 10.0.0.0/16 contenant six subnets repartis sur deux AZ :

Type de subnet	CIDR	AZ	Role
Public x2	10.0.1.0/24, 10.0.2.0/24	1a / 1b	ALB, NAT Gateway
Prive app x2	10.0.11.0/24, 10.0.12.0/24	1a / 1b	Taches ECS Fargate
Prive data x2	10.0.21.0/24, 10.0.22.0/24	1a / 1b	RDS PostgreSQL

Une Internet Gateway donne l'accès entrant/sortant aux subnets publics. Une NAT Gateway (placee dans un subnet public) permet aux taches ECS des subnets privees de joindre Internet en sortie (pull d'image ECR, accès a Secrets Manager et CloudWatch) sans être exposees en entree. Les tables de routage separent le trafic public (via IGW) du trafic prive (via NAT).

3.2 Base de donnees (module rds)

Parametre	Valeur
Moteur	PostgreSQL 16.9
Instance	db.t3.micro
Stockage	20 Go gp3, chiffre (storage_encrypted = true)
Acces public	Non
Multi-AZ	Non (staging - choix de cout)
Sauvegardes	Retention 7 jours
Mot de passe	Gere automatiquement par Secrets Manager (manage_master_user_password)

Le mot de passe maitre n'est jamais écrit en clair : RDS le genere et le stocke dans un secret dedie. Le schema applicatif (tables users, items, reservations + index) a été applique apres deploiement via ECS Exec (voir section 6).

3.3 Secrets (module secrets)

Le secret JWT (chaîne aleatoire de 48 caracteres) utilise pour signer les jetons d'authentification est genere par Terraform et stocke dans Secrets Manager (boiteavoisins/staging/jwt-secret). Aucun secret n'apparaît dans le code ni dans les variables d'environnement en clair du depot.

3.4 Repartition de charge (module alb)

Un Application Load Balancer public repartit le trafic HTTP sur les taches ECS. Il est associe a un Target Group de type IP (obligatoire pour Fargate en mode reseau awsvpc) dont le health check interroge /api/health (code 200 attendu). Le listener HTTP ecoute sur le port 80 et transmet au Target Group.

3.5 Compute (module ecs)

Le coeur applicatif est un service ECS Fargate :

- Cluster avec Container Insights active (metriques detaillees).
- Task definition Fargate (256 CPU / 512 Mo), image tiree depuis ECR.
- Service maintenant 2 taches reparties sur les deux subnets prives (haute disponibilite sur 2 AZ).
- Auto Scaling par suivi de cible (CPU 70 %), de 2 a 4 taches, pour la scalabilite.
- Circuit breaker de deploiement avec rollback automatique en cas d'echec.

Injection de la configuration : les variables non sensibles (hote, port, base, utilisateur) sont passees en clair ; le mot de passe RDS (PGPASSWORD) et le secret JWT sont injectes a l'execution depuis Secrets Manager, jamais stockes dans l'image.

3.6 Frontend (module s3-frontend)

Le frontend React compile est heberge sur un bucket S3 en mode website statique. L'application appelle l'API via le DNS de l'ALB. Le fallback SPA (toute route inconnue renvoyee vers index.html) est assure nativement par la configuration website du bucket (error_document).

Note d'architecture : la cible initiale etait une distribution CloudFront (HTTPS, CDN, OAC) routant /api vers l'ALB pour eviter le mixed-content. CloudFront s'etant revele integrelement bloque sur le compte Learner Lab (voir section 5), le repli S3 website a ete retenu. Le front et l'API etant tous deux servis en HTTP, il n'y a pas de probleme de contenu mixte.

3.7 Observabilite et supervision

- CloudWatch Logs : les journaux applicatifs des taches sont centralises dans /ecs/boiteavoisins-staging-backend (retention 7 jours).
- Container Insights : metriques CPU / memoire / reseau au niveau cluster et service.
- Alarme CloudWatch : declenchee si l'utilisation CPU du service depasse 80 % (2 periodes d'une minute).
- Topic SNS : cible des notifications de l'alarme (point d'extension pour un envoi email).

4. Choix techniques et justifications

Decision	Justification
ECS Fargate plutot qu'EC2	Pas de gestion de serveurs ni de patching ; facturation a l'usage ; scaling simple. Adapte a une API stateless.
2 taches + Auto Scaling	Resilience (perte d'une AZ ou d'une tache sans coupure) et scalabilite horizontale automatique selon la charge CPU.
RDS managed master password	Le mot de passe n'existe nulle part en clair ; rotation et stockage delegues a Secrets Manager.
Injection des secrets dans ECS	Separation stricte configuration / secrets ; l'image Docker reste neutre et reutilisable entre environnements.
SG references croisees	Least privilege : on autorise un Security Group source, pas une plage CIDR large. L'intention de securite est explicite et maintenable.
Remote state S3 + verrou DynamoDB	State partageable, versionne et chiffre ; verrou pour eviter les applications concurrentes.
Circuit breaker + rollback	Un deploiement defaillant revient automatiquement a la version stable precedente, sans intervention.

5. Contraintes du compte Learner Lab et contournements

Le compte AWS Academy Learner Lab impose des restrictions spécifiques. Les contournements suivants ont été nécessaires et constituent des choix techniques assumés.

5.1 Roles IAM : utilisation du LabRole

La création de rôles IAM (iam:CreateRole) est interdite sur le Learner Lab. Le module ECS ne crée donc aucun rôle : il référence le rôle pré-provisionné LabRole (résolu via une data source) comme rôle d'exécution et rôle de tâche. Le LabRole dispose des permissions nécessaires (pull ECR, lecture Secrets Manager, écriture CloudWatch Logs).

5.2 CloudFront indisponible

Toutes les actions CloudFront sont refusées sur ce compte (y compris ListDistributions, CreateDistribution, CreateOriginAccessControl, CreateFunction). L'architecture frontend initialement prévue (CloudFront + OAC + routage /api vers l'ALB) a donc été remplacée par un hébergement S3 website statique. Conséquence positive : front et API étant tous deux en HTTP, le problème de contenu mixte ne se pose pas.

5.3 Comptes isolés

Chaque membre disposant de son propre compte sandbox, le partage de state distant et de réseau entre comptes n'était pas possible. Toute l'infrastructure a donc été déployée dans un seul compte. Un bootstrap Terraform dédié (bucket S3 + table DynamoDB) a été créé dans ce compte pour héberger le state distant.

5.4 Credentials temporaires

Les identifiants du Learner Lab sont temporaires (jeton de session STS) et expirent régulièrement. Ils doivent être rafraîchis dans ~/.aws/credentials à chaque reprise de session ; aucun identifiant n'est versionné (couvert par .gitignore).

6. Procedure de deployment

Le deployment complet, en partant d'un compte vierge, suit l'ordre ci-dessous.

6.1 Bootstrap du state distant

```
cd infra/bootstrap
terraform init
terraform apply -var="project_id=<identifiant-unique>"
```

Cree le bucket S3 (versionne, chiffre) et la table DynamoDB de verrou. Le nom du bucket est ensuite renseigne dans infra/envs/staging/backend.tf.

6.2 Reseau, secrets, base de donnees, ALB

```
cd infra/envs/staging
terraform init -reconfigure
terraform apply -target=module.vpc -target=module.secrets \
  -target=module.sg_ecs -target=module.rds -target=module.alb
```

La creation du RDS prend une dizaine de minutes (etape la plus longue).

6.3 Image Docker et registre ECR

```
aws ecr create-repository --repository-name boiteavoisins-backend \
  --image-scanning-configuration scanOnPush=true
aws ecr get-login-password | docker login --username AWS --password-stdin <ECR>
docker build --platform linux/amd64 -t boiteavoisins-backend:latest .
docker tag ... && docker push <ECR>/boiteavoisins-backend:latest
```

L'URI de l'image (specifique au compte) est renseignee dans un fichier terraform.tfvars non versionne.

6.4 Service ECS

```
aws iam create-service-linked-role --aws-service-name ecs.amazonaws.com
terraform apply -target=module.ecs
```

Le service démarre 2 taches ; verification de leur sante via le Target Group de l'ALB et un appel a /api/health.

6.5 Application du schema SQL (via ECS Exec)

```
aws ecs execute-command --cluster boiteavoisins-staging \
  --task <TASK_ID> --container backend --interactive --command "/bin/sh"
# dans le conteneur :
apk add --no-cache postgresql-client
psql "sslmode=require" -f db/schema.sql
```

Les variables de connexion (PGHOST, PGUSER, PGPASSWORD, PGDATABASE) etant deja injectees dans le conteneur, psql se connecte automatiquement. Le sslmode=require est impose car RDS force le SSL.

6.6 Frontend

```
terraform apply -target=module.frontend
cd frontend
echo "VITE_API_URL=http://<DNS_ALB>" > .env
npm install && npm run build
aws s3 sync dist/ s3://<bucket-frontend> --delete
```

6.7 Validation finale

Verification de bout en bout : creation de compte depuis le navigateur (front -> ALB -> ECS -> RDS), connexion, navigation. Un terraform plan final confirme qu'aucun changement n'est en attente (infrastructure conforme au code).

7. Procedure de rollback

Deux niveaux de retour arriere sont prevus.

7.1 Rollback applicatif automatique

Le service ECS est configure avec un circuit breaker (`deployment_circuit_breaker { enable = true, rollback = true }`). Si un nouveau deploiement ne parvient pas a stabiliser ses taches (echec des health checks), ECS revient automatiquement a la derniere task definition saine, sans intervention.

7.2 Rollback manuel vers une version anterieure

Pour revenir manuellement a une version precedente de l'image, on redeploie en ciblant l'ancienne task definition :

```
aws ecs update-service --cluster boiteavisins-staging \  
--service boiteavisins-staging-backend \  
--task-definition boiteavisins-staging-backend:<REVISION_PRECEDENTE>
```

7.3 Rollback de l'infrastructure

Le state Terraform etant versionne (bucket S3 avec versioning), une modification d'infrastructure indésirable peut être annulée en restaurant une version antérieure du state, ou en re-appliquant le code de la revision Git précédente.

8. Variables d'environnement

8.1 Backend (injectees dans la task ECS)

Variable	Source	Role
PORT	En clair (4000)	Port d'ecoute de l'API
PGHOST / PGPORT	En clair (endpoint RDS)	Hote et port de la base
PGDATABASE / PGUSER	En clair	Base et utilisateur
PGSSL	En clair (true)	Active le SSL vers RDS
PGPASSWORD	Secrets Manager (secret RDS)	Mot de passe base (jamais en clair)
JWT_SECRET	Secrets Manager (secret JWT)	Cle de signature des jetons

8.2 Frontend (au build)

Variable	Valeur	Role
VITE_API_URL	http://<DNS_ALB>	URL de base de l'API appelee par le front

9. Limites connues et ameliorations possibles

- HTTPS : en l'absence de CloudFront et de nom de domaine, le trafic est en HTTP. Amelioration : certificat ACM + listener HTTPS sur l'ALB, ou CloudFront sur un compte non restreint.
- NAT Gateway unique : un seul NAT (staging) constitue un point de defaillance par AZ. En production, un NAT par AZ ameliorerait la resilience.
- RDS mono-AZ : choix de cout en staging. Multi-AZ recommande en production pour la bascule automatique.
- Stockage des photos : les uploads sont ecrits sur le systeme de fichiers ephemere du conteneur. Amelioration : externaliser vers un bucket S3.
- Notifications : le topic SNS est cree mais sans abonnement email ; il suffit d'ajouter une souscription pour recevoir les alertes.

10. Conclusion

L'infrastructure deployee fournit une chaine complete, automatisee et reproductible via Terraform : un reseau segmente multi-AZ, une base de donnees managee et chiffree, un compute conteneurise resilient et auto-scalable derriere un load balancer, une gestion centralisee des secrets et une supervision via CloudWatch. Les principes de moindre privilege sont appliques a chaque flux reseau.

Les contraintes du compte pedagogique (interdiction de creer des roles IAM, CloudFront indisponible, comptes isoles, credentials temporaires) ont ete contournes par des choix documentes (LabRole, hebergement S3 website, deploiement mono-compte), sans compromettre la coherence ni la securite de l'ensemble. L'application est accessible et fonctionnelle de bout en bout, du navigateur jusqu'a la base de donnees.